

Publish Decision Brief — n8n-nodes-baramundi-management-solution

Audience: Senior Product Manager, baramundi software GmbH **Purpose:** Everything you need to decide how and what to publish **Date:** 2026-04-01
Status: Awaiting PM decisions (marked with ?)

1. What Are We Publishing?

An **n8n community node** that lets n8n workflows talk directly to baramundi Management Suite (bMS) via the bConnect REST API — without any custom code.

What it enables for customers

An IT administrator using n8n can build workflows that:

- Query and manage endpoints (Windows, macOS, Linux, Android, iOS, Network)
- Trigger and monitor job executions
- Deploy software, manage bundles, track installed software
- Read compliance and vulnerability data
- Manage Active Directory objects, org units, and group memberships
- Administer server infrastructure, security profiles, and API keys
- Handle variables, maintenance windows, kiosk releases, and more

API coverage

bMS Version	Operations Implemented	Coverage
25R2	251 / 251	100%
26R1	280 / 280	100%

Every single bConnect API endpoint is covered. This is not a proof-of-concept — it is a complete integration.

2. What is npm and Why Does It Matter?

npm in plain terms

npm (Node Package Manager) is the world's largest software registry for JavaScript/TypeScript packages. It lives at npmjs.com. Anyone can publish a package there; anyone can install it.

Think of it as an **app store for developer tools and integrations** — but fully automated and command-line driven.

Why npm is the only viable distribution path

n8n's community node system is **built on top of npm**. This is not an implementation detail — it is the architecture.

When an n8n administrator wants to install a community node, they go to:

```
n8n UI → Settings → Community Nodes → Install
```

They type in a package name (e.g. `n8n-nodes-baramundi-management-solution`) and click Install. n8n then fetches that package from npmjs.com and loads it into the running n8n instance.

There is no other installation path for end users. Without npm publication, administrators would have to manually copy files into the n8n filesystem — which is impractical, unsupported, and breaks on n8n upgrades.

What "publishing" means in practice

Running `npm publish` uploads a package to npmjs.com. From that moment:

- The package is publicly searchable at npmjs.com
- Any n8n user in the world can install it in ~30 seconds
- It is indexed by search engines and n8n's own community node discovery

3. What Gets Published — And What Doesn't

The `files` field in `package.json` controls exactly what is uploaded:

```
"files": ["dist", "LICENSE.md", "README.md", "CHANGELOG.md"]
```

Included	Not Included
<code>dist/</code> — compiled JavaScript (the runnable node)	TypeScript source code (<code>nodes/</code> , <code>credentials/</code>)
<code>LICENSE.md</code> — MIT license text	Tests (<code>test/</code>)
<code>README.md</code> — user-facing documentation	Dev tooling (<code>eslint.config.js</code> , <code>tsconfig.json</code>)
<code>CHANGELOG.md</code> — version history	Internal planning docs (<code>Tasks.md</code> , <code>tasks_todo.md</code> , etc.)

Before upload, the build step runs automatically (`prepublishOnly: npm run build`), ensuring `dist/` is always freshly compiled from the current source.

The TypeScript source is not published. Customers and users only receive the compiled output.

4. The Three Things That Make It a Valid n8n Node

n8n enforces three hard requirements. All three are currently satisfied:

Requirement	Current Value	Status
Package name starts with <code>n8n-nodes-</code>	<code>n8n-nodes-baramundi-management-solution</code>	✓
<code>keywords</code> includes <code>n8n-community-node-package</code>	Present	✓
<code>n8n</code> field lists compiled node + credential files	Present and correct	✓

If any of these is missing, n8n silently refuses to install the package.

5. The Three Supporting Files

LICENSE.md ✓ Exists

Currently declares **MIT License**, copyright `baramundi software GmbH`.

What MIT means: - Anyone can use, copy, modify, and redistribute the software — including commercially - They must include the copyright notice - baramundi has no liability for how it is used

? Decision required: Is MIT the right license? Options:

License	Implication
MIT (current)	Maximum adoption. Anyone can fork, redistribute, embed.
Apache 2.0	Similar openness, adds explicit patent grant — common for corporate OSS.
Proprietary / EULA	Cannot be published as an n8n community node — n8n requires OSS license.

MIT is standard for n8n community nodes. Apache 2.0 is also acceptable. Proprietary is not compatible with community node distribution.

README.md Exists

The README is the **first thing a user sees** on npmjs.com and in the n8n community node browser. It currently covers:

- Features overview
- Installation instructions (both development and production)
- Credentials configuration
- bMS version targeting (25R2 vs 26R1)
- Available resources and operations
- Example workflows
- Project structure
- Contributing guidelines
- License

? Decision required: Review before publish. Key questions: - Does the branding reflect official baramundi product positioning? - Are the example workflows representative of real customer use cases? - Should it reference official baramundi support channels? - Is the "Contributing" section appropriate for an externally published package?

CHANGELOG.md Exists

Documents every version from 0.1.0 through 0.4.1. Follows the [Keep a Changelog](#) standard and [Semantic Versioning](#).

npm users and n8n administrators use the changelog to evaluate whether a version bump is safe to install. A well-maintained changelog builds trust.

Current latest entry: [0.4.1] - 2026-04-01 — security hardening patch.

6. Versioning — 0.4.1 or 1.0.0?

How Semantic Versioning works

Version numbers follow the format `MAJOR.MINOR.PATCH` :

Part	When to bump	Example
PATCH	Bug fixes, security patches, no new features	0.4.0 → 0.4.1
MINOR	New features, backwards compatible	0.4.1 → 0.5.0
MAJOR	Breaking changes, or declaring public stability	0.x.x → 1.0.0

The leading `0` in `0.x.x` is a conventional signal that the package is **not yet stable** — breaking changes may happen at any minor version.

Current version: `0.4.1`

Publishing as `0.4.1` signals: - Pre-release / not yet declared stable - Internal team retains freedom to make breaking changes in `0.5.x`, `0.6.x` etc. - Sophisticated users will know to pin their version

Publishing as `1.0.0` signals: - This is stable, production-ready software - Breaking changes will only come in `2.0.0` - Creates an implicit support commitment to users who adopt it

? Decision required: Given that this is the first public release and coverage is 100%, is this product-ready enough to call `1.0.0` ? Or is `0.4.1` the right signal while the community matures?

7. Visibility and Ownership

Public vs scoped package

Option	npm name	Implication
Public (current)	<code>n8n-nodes-baramundi-management-solution</code>	Globally discoverable, anyone can install
Scoped public	<code>@baramundi/n8n-baramundi-management-solution</code>	Same visibility, but namespaced under a baramundi npm org
Scoped private	<code>@baramundi/n8n-nodes-... (private)</code>	Not installable by n8n community node mechanism — defeats the purpose

Scoped public (`@baramundi/...`) requires creating a **baramundi npm organization** on npmjs.com. It has the advantage of clearly signalling official baramundi authorship and prevents name squatting on the `@baramundi` namespace.

? Decision required: Should this be published under a `@baramundi` npm org scope, or as a plain public package?

Name availability

? Action required before publish: Run the following to check if the name is already taken:

```
npm info n8n-nodes-baramundi-management-solution
```

If it returns a 404, the name is available. If it returns a package, you must choose a different name or contact npm support.

Who owns the npm account?

Publishing requires being logged in to an npm account that has publish rights to the package.

? Decision required: - Which npm account publishes this? (Individual developer account, or a `baramundi` organization account?) - Who has the credentials? - Who is authorized to publish future updates (security patches, new bMS versions)? - What happens if that person leaves baramundi?

Recommendation: create an **npm organization** (`baramundi`) and publish under it, so ownership is tied to the company, not an individual.

8. GitHub Repository

The `package.json` currently references:

```
"homepage": "https://github.com/baramundi-software/n8n-nodes-baramundi#readme",  
"bugs":     "https://github.com/baramundi-software/n8n-nodes-baramundi/issues",  
"repository": "https://github.com/baramundi-software/n8n-nodes-baramundi.git"
```

These URLs will appear on the npmjs.com package page and are clicked by users evaluating the package.

? Decision required: - Does the GitHub repository `baramundi-software/n8n-nodes-baramundi` exist? If not, these are dead

links — which damages trust and makes it impossible for users to report issues. - Should the repo name be updated to match the new package name (`n8n-nodes-baramundi-management-solution`)? - Should the repo be public (recommended for a published community node) or private?

Recommendation: Create the public GitHub repo, push the source code, then publish to npm. The two should go live together.

9. Risks and Considerations

Once published, it's permanent

npm allows "unpublishing" a package within **72 hours** of first publish. After that, the package is permanently on the registry. Even if you unpublish, the name is reserved and the version cannot be re-used.

Implication: Do not publish a test version or a placeholder. The first publish should be the real release.

Community support expectations

Once a package is publicly available on npmjs.com, the n8n community will find it, install it, and open issues. There will be: - Bug reports - Feature requests - Questions about compatibility with new bMS versions - Requests for new bConnect API endpoints as baramundi adds them

? Decision required: Who handles community issues? Is there a support policy? Should the README state that this is community-supported vs. officially supported by baramundi?

MIT license obligations

Under MIT, any user can: - Fork the repo and publish their own version - Embed the connector in commercial products - Redistribute modified versions

They must keep the copyright notice. baramundi cannot prevent this.

bMS version lifecycle

When baramundi releases a new bMS version with new bConnect APIs, the connector will be out of date until updated. Users will notice and ask. Plan for a maintenance process aligned with bMS release cycles.

10. The Actual Publish Steps

Once all decisions above are resolved, the publish process is:

```
# 1. Log in to the npm account that will own the package
npm login

# 2. Verify the package name is available
npm info n8n-nodes-baramundi-management-solution

# 3. Verify a clean build
npm run build

# 4. DRY RUN – inspect every file that will be uploaded before anything goes live
npm publish --dry-run

# 5. Go live
npm publish
```

The `--dry-run` step is critical. It prints a complete list of every file that would be included in the published package, the total size, and the exact version being published — without actually uploading anything. Review this output carefully before running the final `npm publish`.

11. Go / No-Go Checklist

Before running `npm publish`, the following must all be confirmed:

#	Item	Owner	Status
1	License choice confirmed (MIT or Apache 2.0)	PM	?
2	README reviewed and approved for external audience	PM	?
3	CHANGELOG reviewed — version history is acceptable to publish	PM	?
4	Version number decided (0.4.1 or 1.0.0)	PM	?
5	Package name availability confirmed (<code>npm info ...</code>)	Dev	?
6	npm account / org created and credentials held securely	IT/Dev	?
7	Scoped (<code>@baramundi/</code>) vs unscoped package decided	PM	?

#	Item	Owner	Status
8	GitHub repository exists and is public	Dev	?
9	GitHub repo URLs in <code>package.json</code> match actual repo name	Dev	?
10	Community support / issue handling process defined	PM	?
11	<code>npm publish --dry-run</code> output reviewed and approved	Dev + PM	?

All 11 items green → run `npm publish`.

12. Open Questions Summary

Question	Impact
MIT or Apache 2.0 license?	Legal / redistribution rights
Version 0.4.1 or 1.0.0?	User expectations, support commitment
Scoped <code>@baramundi/</code> or plain package name?	Brand identity, npm org setup required
Which npm account owns publication rights?	Long-term maintainability
Does the GitHub repo exist?	Dead links, user trust, issue tracking
Who handles community support after publish?	Support workload, reputation risk
Is the README approved for external audiences?	Brand / product positioning